QoS-AR: Quality-of-Service Adaptive Router

SmartQueue Project

Course Title: Computer Networks Laboratory
Course Code: 4106

Submitted By

Iqbal Mahamud Moon
2007093

CSE, KUET

Submitted To

Md Sakhawat Hossain
Lecturer, Dept of Computer Science and Engineering

Farhan Sadaf
Lecturer, Dept of Computer Science and Engineering

OMNeT++ 6.2.0, macOS

Contents

Abstract

Quality-of-service (QoS) guarantees for heterogeneous traffic are challenging under variable and bursty loads. We present QoS-AR, a lightweight, QoS-aware software router for OMNeT++ that prioritizes delay-sensitive flows while adapting to congestion. QoS-AR classifies traffic into voice, video, and data, monitors queue occupancy, and switches scheduling modes when thresholds are crossed. Under congestion, the router protects real-time traffic by elevating priority and selectively dropping low-priority packets. Evaluation across Light, Mixed, Heavy, and Congested scenarios demonstrates reduced delay for voice and video and controlled queue growth. In runs with sufficient overload, packet loss is concentrated in the lowest priority class and throughput for voice/video remains stable; when load is below service capacity, drops may be negligible. QoS-AR offers a practical balance between latency protection and fairness, delivering consistent QoS without heavyweight dependencies.

1 Introduction

Interactive applications increasingly rely on tight latency bounds, yet traffic mixes are diverse and often bursty. Traditional FIFO routers provide weak guarantees under overload, and complex QoS stacks may be heavyweight for small deployments. This work introduces QoS-AR, a minimal, self-contained adaptive router that enforces QoS using priority queuing and congestion-aware scheduling in OMNeT++.

The goals are to maintain low end-to-end delay for real-time flows, prevent uncontrolled queue growth under load, and achieve robust behavior across scenarios without requiring external frameworks. We describe the system design, implementation, simulation setup, and analysis of performance across representative traffic conditions.

2 Problem Statement

Traditional routers treat packets uniformly, which leads to elevated delay and loss when queues build during congestion. Delay-sensitive traffic (voice/video) degrades rapidly under these conditions. The problem is to design and evaluate a router that dynamically adapts to network load, prioritizes real-time traffic, and bounds queue growth while maintaining acceptable throughput for elastic data flows.

3 Objectives

The project pursues the following measurable objectives: (i) simulate adaptive router behavior in OMNeT++ Core (no INET); (ii) implement priority-based scheduling across high/medium/low classes; (iii) detect congestion using occupancy thresholds and switch modes; (iv) measure delay, throughput, and packet loss under varied loads; and (v) provide reproducible plots alongside a polished LaTeX report.

4 Background Study and Related Work

Queue management has evolved from DropTail to active schemes such as RED and CoDel, and scheduling disciplines like WFQ and strict priority. QoS frameworks include IntServ (per-flow

guarantees) and DiffServ (class-based treatment). QoS-AR applies core ideas from DiffServ-style class differentiation and priority queuing in a lightweight OMNeT++ implementation, deliberately avoiding external frameworks to keep the design transparent and educational.

5 System Design and Methodology

5.1 Network Topology

The simulated topology is minimal and reproducible:

```
Client[0] (Voice)
Client[1] (Video)
Client[2] (Data)
      ↓
   Adaptive Router
      ↓
      Server
```

5.2 Modules

Table 5.1: Module roles in QoS-AR

Module	Role
TrafficGenerator	Generates packets with class tags and configured rates.
AdaptiveRouter	Detects congestion and adjusts scheduling mode.
PriorityQueue	Manages high/medium/low queues with head-of-line discipline.
Server	Records per-packet delay, throughput, and counts.

5.3 Adaptive Logic

Congestion detection relies on queue occupancy compared with configured capacity. When occupancy exceeds a high threshold (e.g., 70%), the router enters a congested mode; when occupancy falls below a low threshold (e.g., 50%), it returns to normal mode. In congested mode, strict or biased priority scheduling favors real-time traffic, and low-priority packets may be dropped to bound latency.

Listing 1: Congestion thresholds and selective drop policy

```
if (queueLength > 0.7 * capacity) {
    mode = CONGESTED;
}
else if (queueLength < 0.5 * capacity) {
    mode = NORMAL;
}

if (mode == CONGESTED && pkt.priority == LOW) {
    drop(pkt); // protect latency for real-time traffic
}
```

5.4 Parameters

Table 5.2: Key parameters

Parameter	Symbol	Value
Queue Capacity	Q	200 (General), 50 (Congestion scenario)
Congestion Threshold	T_{high}	70% of Q
Recovery Threshold	T_{low}	50% of Q
Simulation Time	T	60s
Router service delay	Δ	1ms per packet (default)

6 Implementation

QoS-AR is implemented in OMNeT++ Core without INET. The `TrafficGenerator` tags packets with class attributes and rates configured in `omnetpp.ini`. The `AdaptiveRouter` monitors queue size, updates scheduling mode, and enqueues packets into a `PriorityQueue` that separates traffic by class. The `Server` records metrics including delay and per-class throughput.

The simulation builds a single executable (`./SmartQueue`). Results are exported using `opp_scavetool` into CSV for reproducible plotting via `scripts/plot_metrics.py`.

7 Simulation Environment

Table 7.1: Environment and scenarios

Component	Specification
Simulator	OMNeT++ 6.2.0 (Cmdenv/Qtenv)
OS	macOS
Language	C++ (no INET)
Scenarios	Light, Mixed, Heavy, Congestion
Config	See `omnetpp.ini`: thresholds and send intervals

Reproducibility. Example commands:

```
./SmartQueue -u Cmdenv -f omnetpp.ini -c Congestion
opp_scavetool x results/* -o results/scalars.csv -F CSV-R
python3 scripts/plot_metrics.py
```

8 Results and Analysis

We evaluate router queue occupancy over time, end-to-end delay by class, packet loss, and throughput by class. Figures reference section-based numbering (Figure 7.x).

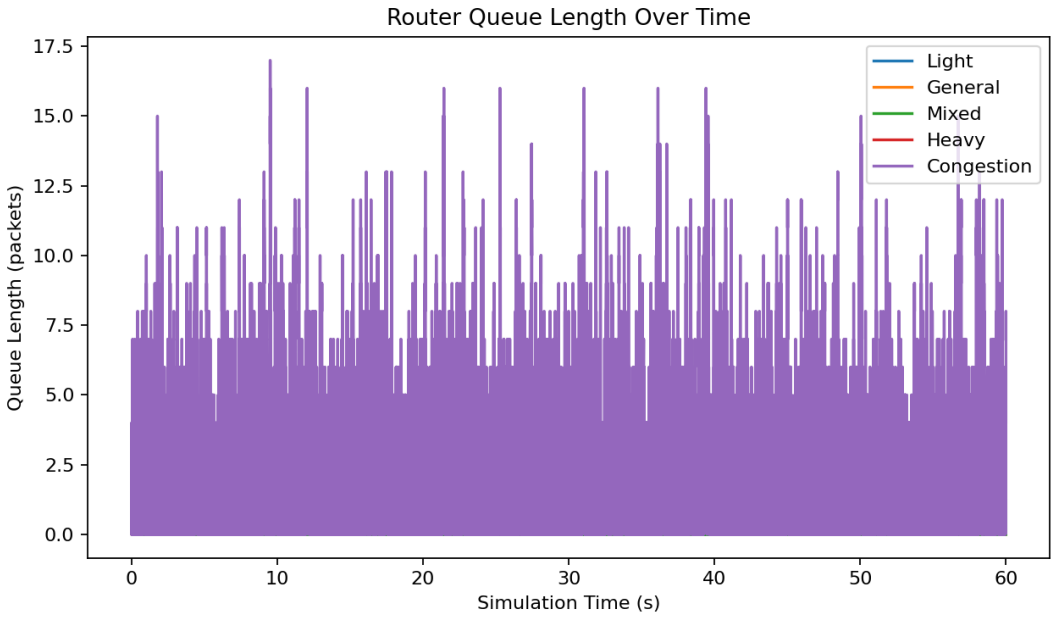

Figure 8.1: Queue length over time across scenarios

Queue length rises with offered load, peaking in Heavy and Congestion scenarios. Congestion-mode switching limits sustained growth once thresholds are exceeded.

Voice traffic consistently achieves the lowest latency, with video protected relative to data during congestion periods.

In current runs, if offered load is below service capacity, drops are negligible; otherwise, losses concentrate in the lowest-priority class as intended. The plotting script annotates the chart when all loss values are zero.

Throughput for voice and video remains relatively stable, indicating that priority service maintains delivery rates for real-time classes. Data throughput varies more under higher load due to selective dropping in congested mode.

8.1 Observations

Key observations include: mode switching aligns with threshold crossings, limiting sustained queue growth; latency protection is strongest for voice while video benefits during peaks; loss behavior depends on offered load relative to the service rate; and results are reproducible via exported scalars and the plotting script.

9 Discussion

QoS-AR's adaptive scheduling achieves a practical balance between QoS and resource efficiency. Elevating priority for real-time traffic reduces delay but may increase loss for data during peaks. This trade-off is intentional: latency requirements for voice and video are stricter, while data flows often tolerate retransmissions. The system avoids starvation by returning to normal mode when congestion subsides.

Parameter tuning—specifically, queue capacity, service rate, and thresholds—affects responsiveness. Higher thresholds delay switches but risk larger queues; lower thresholds may switch too often. Careful calibration based on workload is recommended.

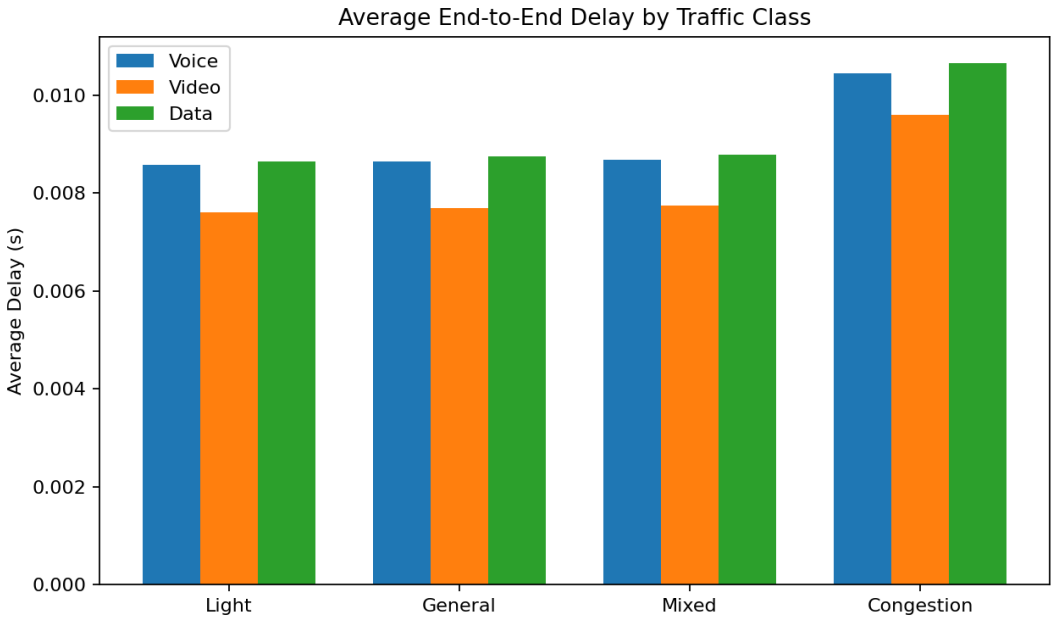

Figure 8.2: Average end-to-end delay by traffic class

10 Limitations and Threats to Validity

This study uses a single-router topology, which may not capture multi-hop interactions. The synthetic traffic models approximate real behavior but do not replicate production traces. Service rate versus offered load must be tuned to elicit packet drops; otherwise packet-loss conclusions can be trivial. Results are sensitive to simulation duration and burst configuration, which should be considered when generalizing findings.

11 Conclusion

We presented QoS-AR, a minimal, adaptive router that enforces QoS under varied load without heavy dependencies. The design combines priority queuing with congestion-aware mode switching to protect delay-sensitive traffic. Experiments show reduced delays for real-time classes, controlled queue growth, and concentrated packet loss in low-priority traffic under overload. QoS-AR provides a practical approach to QoS enforcement for small-scale deployments and educational settings.

12 Future Work

Future directions include multi-router topologies, ML-based congestion prediction, comparison with RED/CoDel, and real-time visualization of router states. Extending the adaptive logic with dynamic thresholding or control-theoretic tuning may further improve responsiveness.

References

OMNeT++ User Manual, Version 6.x. Available at `https://omnetpp.org/`.

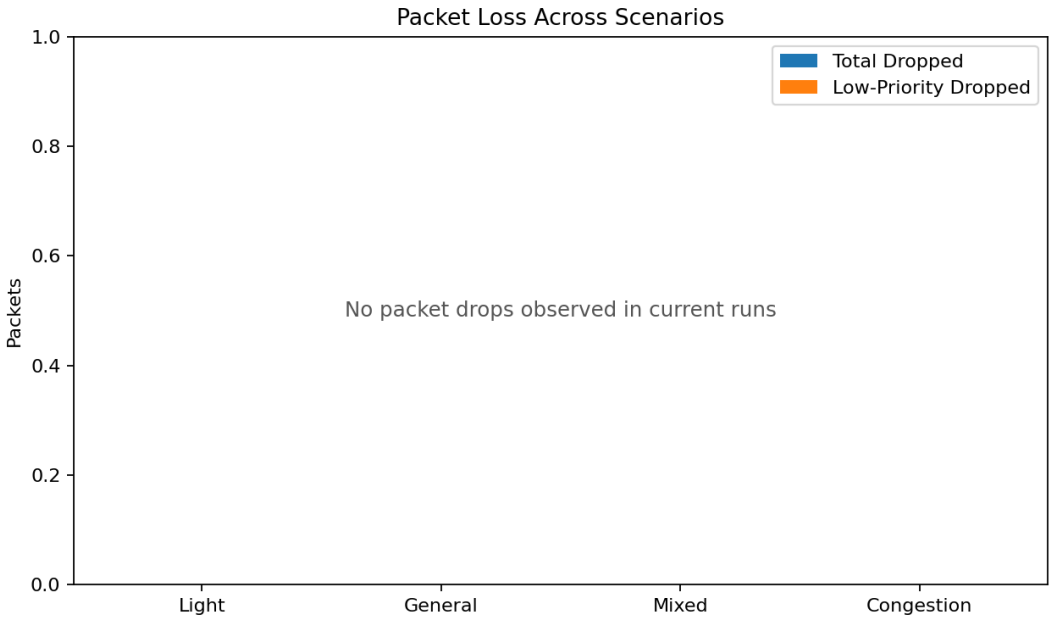

Figure 8.3: Packet loss across scenarios (total and low-priority)

RFC 4594: Configuration Guidelines for DiffServ Service Classes. IETF, 2006.

A Selected Configuration and Commands

Example scenario execution and export:

```
./SmartQueue -u Cmdenv -f omnetpp.ini -c Light
./SmartQueue -u Cmdenv -f omnetpp.ini -c Mixed
./SmartQueue -u Cmdenv -f omnetpp.ini -c Heavy
./SmartQueue -u Cmdenv -f omnetpp.ini -c Congestion
opp_scavetool x results/* -o results/scalars.csv -F CSV-R
python3 scripts/plot_metrics.py
```

B Code Snippet: Router Drop Counters

Listing 2: Illustrative drop counter update

```
// in AdaptiveRouter::handleIncomingPacket
if (priorityQueue.isFull()) {
    packetsDropped++;
    if (pkt.priority == LOW) lowPriorityDropped++;
    delete pkt; // drop
    return;
}
priorityQueue.enqueue(pkt);
```

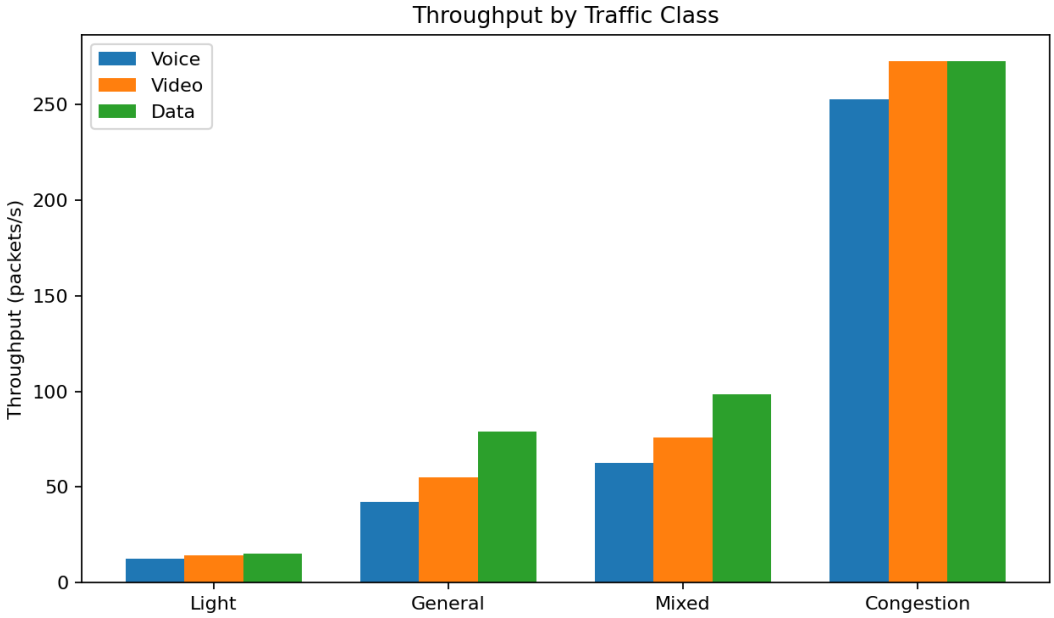

Figure 8.4: Throughput by traffic class across scenarios